

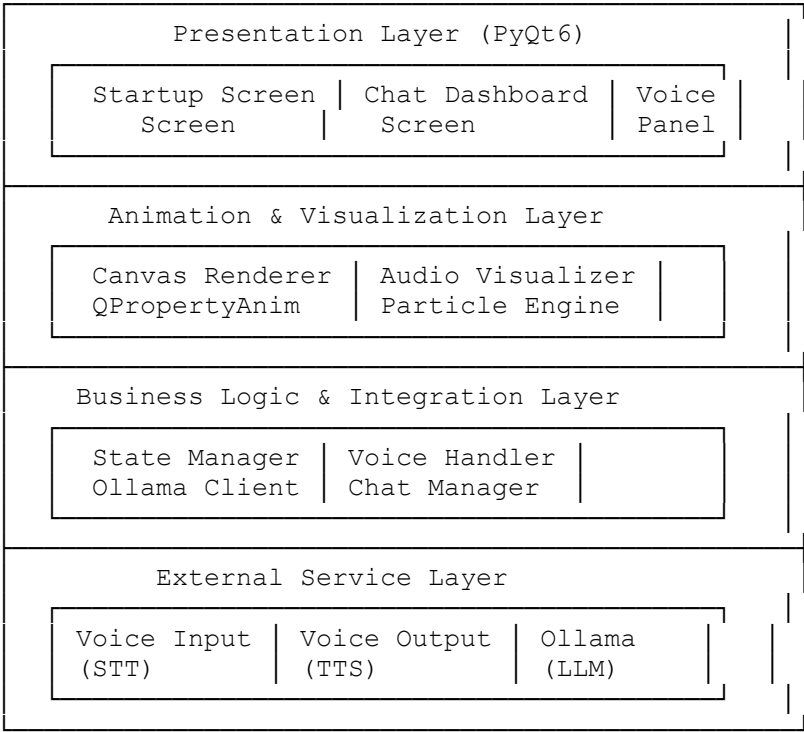
JARVIS HOLOGRAPHIC UI SYSTEM

Architecture & Design Document (Repository-Independent)

1. OVERALL APPLICATION ARCHITECTURE

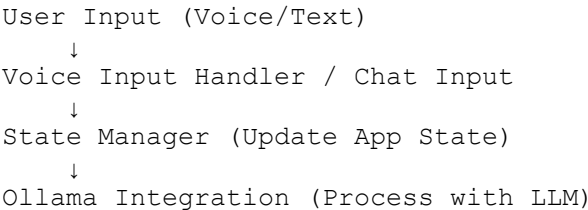
1.1 Architecture Pattern: Layered MVC with State Management

Code



1.2 Data Flow Overview

Code



↓
Response + Voice Output Generation
↓
Audio-Reactive Visualization
↓
UI Update (Animation → Render)
↓
User Output (Voice + Visual Feedback)

2. FOLDER STRUCTURE

Code

```
jarvis_holographic/  
├── main.py                                # Application entry point  
├── config/  
│   ├── __init__.py  
│   ├── constants.py                    # Colors, sizes, animation durations  
│   ├── jarvis_voice_config.py          # JARVIS voice deepening parameters  
│   └── system_config.py                # UI scaling, theme settings  
├── core/  
│   ├── __init__.py  
│   ├── state_manager.py                # Global app state & event system  
│   └── app_context.py                  # Shared context (Ollama, voice)  
├── handlers)  
│   └── event_bus.py                    # Event-driven communication  
├── ui/  
│   ├── __init__.py  
│   ├── main_window.py                  # Root application window  
│   ├── screens/  
│   │   ├── __init__.py  
│   │   ├── startup_screen.py           # JARVIS boot animation  
│   │   ├── mode_selection_screen.py    # Chat vs Voice mode  
│   │   ├── chat_dashboard.py           # Chat UI with reactor  
│   │   └── voice_dashboard.py           # Voice mode with waveforms  
│   ├── widgets/  
│   │   ├── __init__.py  
│   │   ├── arc_reactor.py              # Rotating holographic reactor  
│   │   ├── status_panel.py              # Status indicators  
│   │   ├── waveform_display.py          # Real-time audio waveform  
│   │   ├── equalizer_display.py         # Frequency band visualizer  
│   │   └── particle_system.py           # Holographic particles  
│   └── styles/  
│       ├── __init__.py  
│       ├── theme.py                     # Stylesheet & colors  
│       └── animations.css               # Gradient definitions  
└── animations/  
    ├── __init__.py  
    ├── animation_engine.py              # Central animation coordinator  
    └── property_animations.py            # QPropertyAnimation wrappers
```

	custom_painters.py	# Custom paint events
	transition_effects.py	# Screen transitions & effects
audio/		
	__init__.py	
	voice_handler.py	# Existing voice input/output wrapper
	jarvis_voice_processor.py	# JARVIS voice deepening & TTS params
	audio_visualizer.py	# FFT, waveform extraction
	audio_buffer.py	# Circular audio buffer for real-time
viz		
	audio_reactive_engine.py	# Maps audio → visual properties
integration/		
	__init__.py	
	ollama_client.py	# Ollama integration (existing)
	chat_manager.py	# Chat history & logic (existing)
	date_time_handler.py	# Fixed date/time retrieval
	plugin_interface.py	# Future ElevenLabs integration
utils/		
	__init__.py	
	logger.py	# Logging system
	validators.py	# Input validation
	helpers.py	# Utility functions
	performance_monitor.py	# Debug FPS, render times
resources/		
	fonts/	
	orbitron.ttf	# Sci-fi font
	sounds/	
	startup_beep.wav	
	ui_click.wav	
	images/	
	jarvis_ring.png	# Startup ring reference
tests/		
	__init__.py	
	test_animations.py	
	test_voice_handler.py	
	test_ollama_integration.py	

3. FILE MANIFEST

File Path	Purpose	Priority
main.py	Application entry, initialization	CRITICAL
core/state_manager.py	Global state, event broadcasting	CRITICAL
core/app_context.py	Shared services container	CRITICAL
ui/main_window.py	Root QMainWindow, screen switching	CRITICAL

File Path	Purpose	Priority
ui/screens/startup_screen.py	Boot animation + JARVIS greeting	HIGH
ui/screens/mode_selection_screen.py	Chat/Voice mode picker	HIGH
ui/screens/chat_dashboard.py	Chat interface with reactor	HIGH
ui/screens/voice_dashboard.py	Voice mode with audio viz	HIGH
ui/widgets/arc_reactor.py	Holographic reactor widget	HIGH
ui/widgets/waveform_display.py	Real-time waveform visualization	HIGH
ui/widgets/equalizer_display.py	Frequency-band equalizer viz	HIGH
ui/widgets/particle_system.py	Holographic particle effects	MEDIUM
animations/animation_engine.py	Central animation coordinator	CRITICAL
animations/property_animations.py	QPropertyAnimation helpers	HIGH
animations/custom_painters.py	Custom QPainter implementations	HIGH
audio/voice_handler.py	Wrapper for existing voice I/O	CRITICAL
audio/jarvis_voice_processor.py	JARVIS voice deepening logic	HIGH
audio/audio_visualizer.py	FFT & audio data extraction	HIGH
audio/audio_buffer.py	Ring buffer for real-time data	MEDIUM
audio/audio_reactive_engine.py	Audio → animation mapping	HIGH
integration/date_time_handler.py	System date/time + timezone	HIGH
config/constants.py	All magic numbers & colors	MEDIUM
config/jarvis_voice_config.py	JARVIS voice parameters	HIGH
utils/logger.py	Structured logging	MEDIUM
utils/performance_monitor.py	FPS & profiling tools	MEDIUM

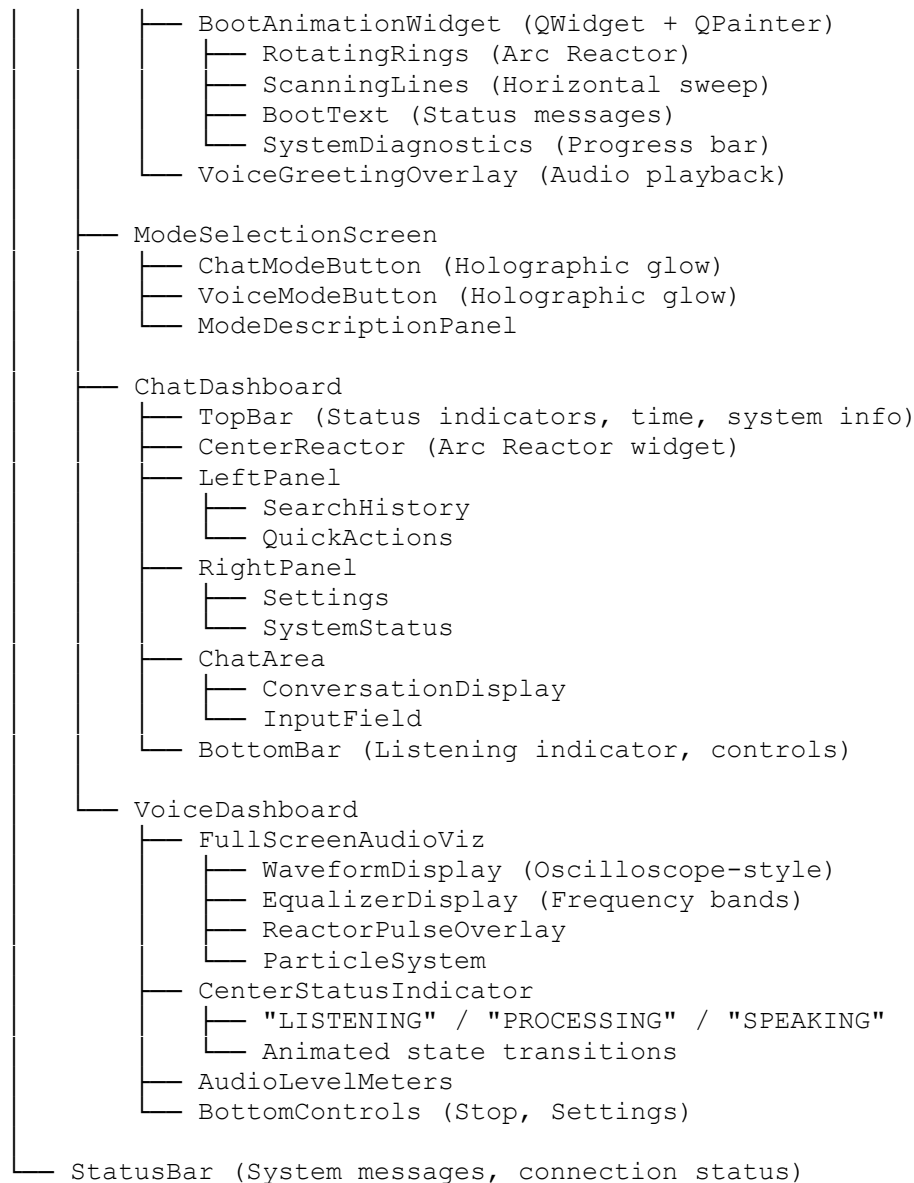
4. UI HIERARCHY

Code

```

QMainWindow (main_window.py)
├── CentralWidget (QStackedWidget) ← Screen switching hub
│   └── StartupScreen

```



5. STARTUP SCREEN ARCHITECTURE

5.1 Startup Flow

Code

Application Launch

↓

Load Configuration (Theme, Voice, Ollama)

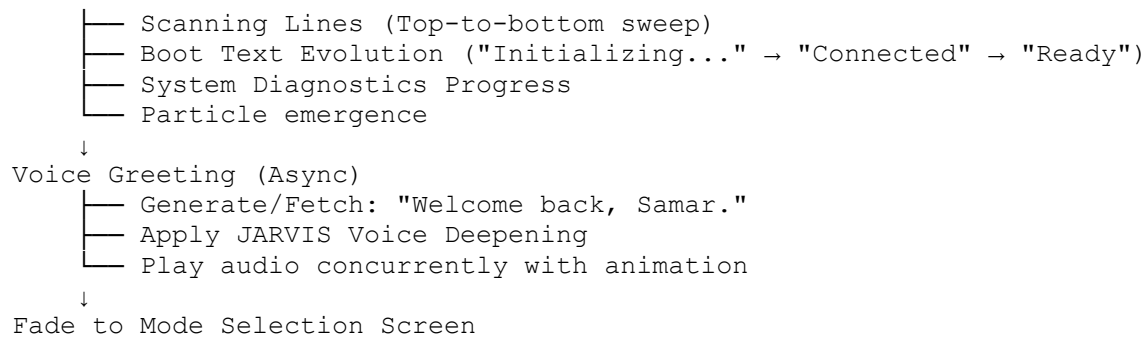
↓

Display StartupScreen

↓

Boot Animation Loop (8-10 seconds)

└─ Rotating Rings (Smooth rotation)



5.2 Animation Components

Component	Duration	Type	Details
Rotating Rings	8s	Continuous	3 concentric rings, varying rotation speeds
Scanning Lines	8s	Cyclic	3-5 horizontal lines sweeping downward, fade effect
Boot Text	8s	Fade sequence	"System Check" → "Initializing..." → "Connected" → "Ready"
Diagnostics Progress	8s	Linear growth	Arc grows from 0° to 270°
Particle Birth	0.5s-4s	Random onset	Particles emerge at ring intersections
Glow Intensity	8s	Pulse	Cyan glow intensity modulates

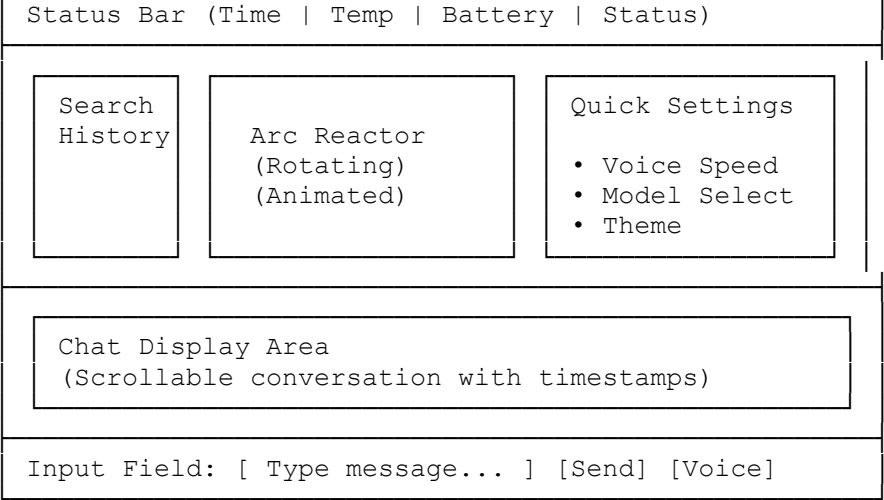
5.3 Rendering Strategy

- **Custom QPainter** for rotating rings and scanning lines
- **Separate QTimer** @ 60 FPS for smooth animation
- **QPropertyAnimation** for opacity fades (text transitions)
- **Audio playback** on separate thread to avoid UI blocking

6. CHAT DASHBOARD ARCHITECTURE

6.1 Layout Structure

Code



6.2 Component Details

Component	Type	Details
Arc Reactor	Custom Widget	Center, always visible, pulses on Ollama response
Search History	QListWidget	Clickable shortcuts to previous queries
Status Panel	Custom	Displays Ollama connection, response time, temperature
Chat Area	QTextEdit	Rich text display with timestamps, user vs JARVIS styling
Input Field	QLineEdit	Text input with "Enter to send"
Voice Button	QPushButton	Switches to voice mode or starts voice input

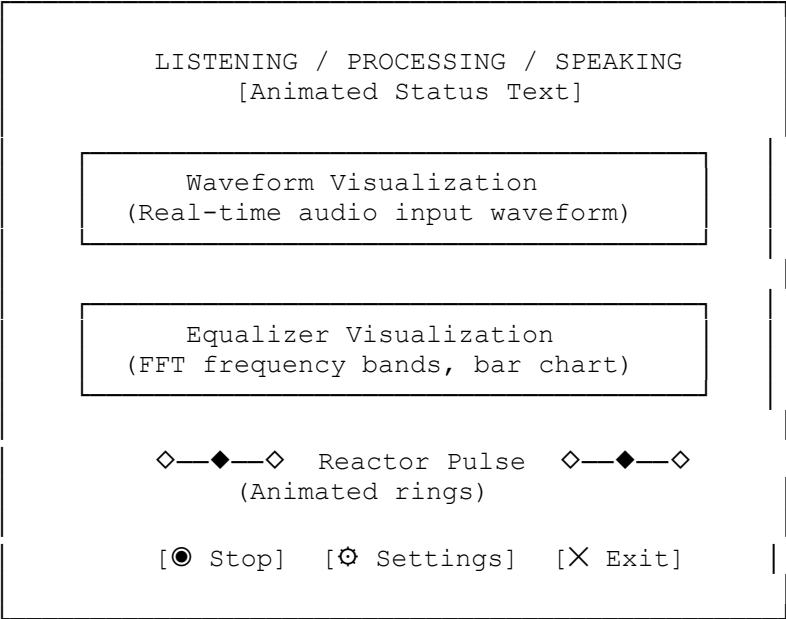
6.3 Animation Behaviors

- **Reactor Pulse:** When Ollama is processing, reactor glows brighter
- **Hover Effects:** Buttons glow on hover (QPropertyAnimation opacity)
- **Text Fade-in:** Chat messages fade in as they appear
- **Panel Slide:** Left/right panels slide in on first interaction
- **Smooth Transitions:** Between states (idle → processing → response)

7. VOICE DASHBOARD ARCHITECTURE

7.1 Full-Screen Voice Interface Layout

Code

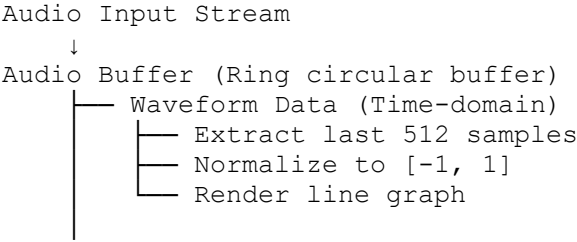


7.2 Voice State Visualization

State	Waveform	Equalizer	Reactor	Text
LISTENING	High amplitude, dynamic	Real-time bands	Steady glow	"LISTENING..."
PROCESSING	Fades out	Dims, processing animation	Pulse speed increases	"PROCESSING..."
SPEAKING	Waveform of JARVIS voice	Frequency response of speech	Bright pulse sync	"SPEAKING..."
IDLE	Flat line	Minimal bars	Slow gentle glow	"READY"

7.3 Audio-Reactive Visualization

Code



- └─ Frequency Data (Frequency-domain via FFT)
 - └─ Apply FFT algorithm
 - └─ Divide into 32 bands (logarithmic scaling)
 - └─ Smooth with exponential averaging
 - └─ Render bar chart
-

8. ANIMATION ENGINE DESIGN

8.1 Central Animation Coordinator

Python

```
class AnimationEngine:
    """
    Centralized animation management for PyQt6.
    - Manages QPropertyAnimation groups
    - Coordinates timing across multiple animations
    - Provides callbacks for custom painting
    - Throttles updates to 60 FPS
    """

    def __init__(self):
        self.animation_group = QParallelAnimationGroup()
        self.custom_animations = {} # For QTimer-based custom paint
        self.fps_limiter = 60 # Target FPS
        self.active_animations = set()

    def register_animation(self, name, animation):
        """Register a QPropertyAnimation for centralized management"""

    def create_fade_transition(self, widget, duration_ms, start_opacity,
end_opacity):
        """Factory for fade transitions"""

    def create_slide_animation(self, widget, start_pos, end_pos, duration_ms,
easing):
        """Factory for slide animations"""

    def create_rotation_animation(self, painter_context, duration_ms,
easing):
        """Custom rotation via QTimer callback"""

    def start_all(self):
        """Start all registered animations"""

    def pause_all(self):
        """Pause all animations"""

    def stop_all(self):
        """Stop and reset all animations"""
```

8.2 Animation Registry

Code

Startup Screen:

- Rotating Rings (Custom QTimer-based)
- Scanning Lines (Custom QTimer-based)
- Text Fade-In (QPropertyAnimation - opacity)
- Progress Arc (QPropertyAnimation - custom angle property)
- Glow Pulse (QPropertyAnimation - color intensity)

Mode Selection:

- Button Hover Glow (QPropertyAnimation - color)
- Button Press Scale (QPropertyAnimation - geometry)
- Screen Fade Transition (QPropertyAnimation - opacity)

Chat Dashboard:

- Reactor Center Glow (QPropertyAnimation - color)
- Reactor Pulse on Response (Custom timing)
- Panel Slide-In (QPropertyAnimation - pos)
- Chat Message Fade-In (QPropertyAnimation - opacity)
- Hover Effects on Buttons (QPropertyAnimation - color)

Voice Dashboard:

- Waveform Real-time Update (Custom QTimer - no animation)
- Equalizer Bars Update (Custom QTimer - smoothed)
- Reactor Pulse Sync (QPropertyAnimation - synchronized with audio)
- Status Text Blink (QPropertyAnimation - opacity)
- Particle Birth/Death (QPropertyAnimation - opacity + pos)

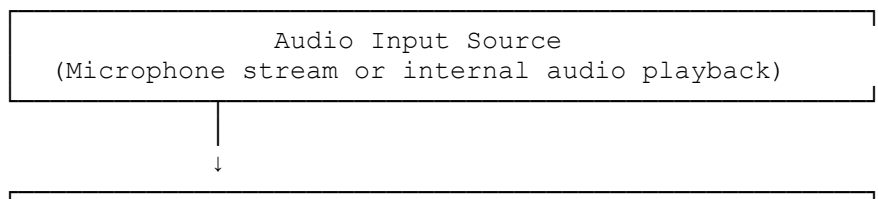
8.3 Performance Optimization Strategy

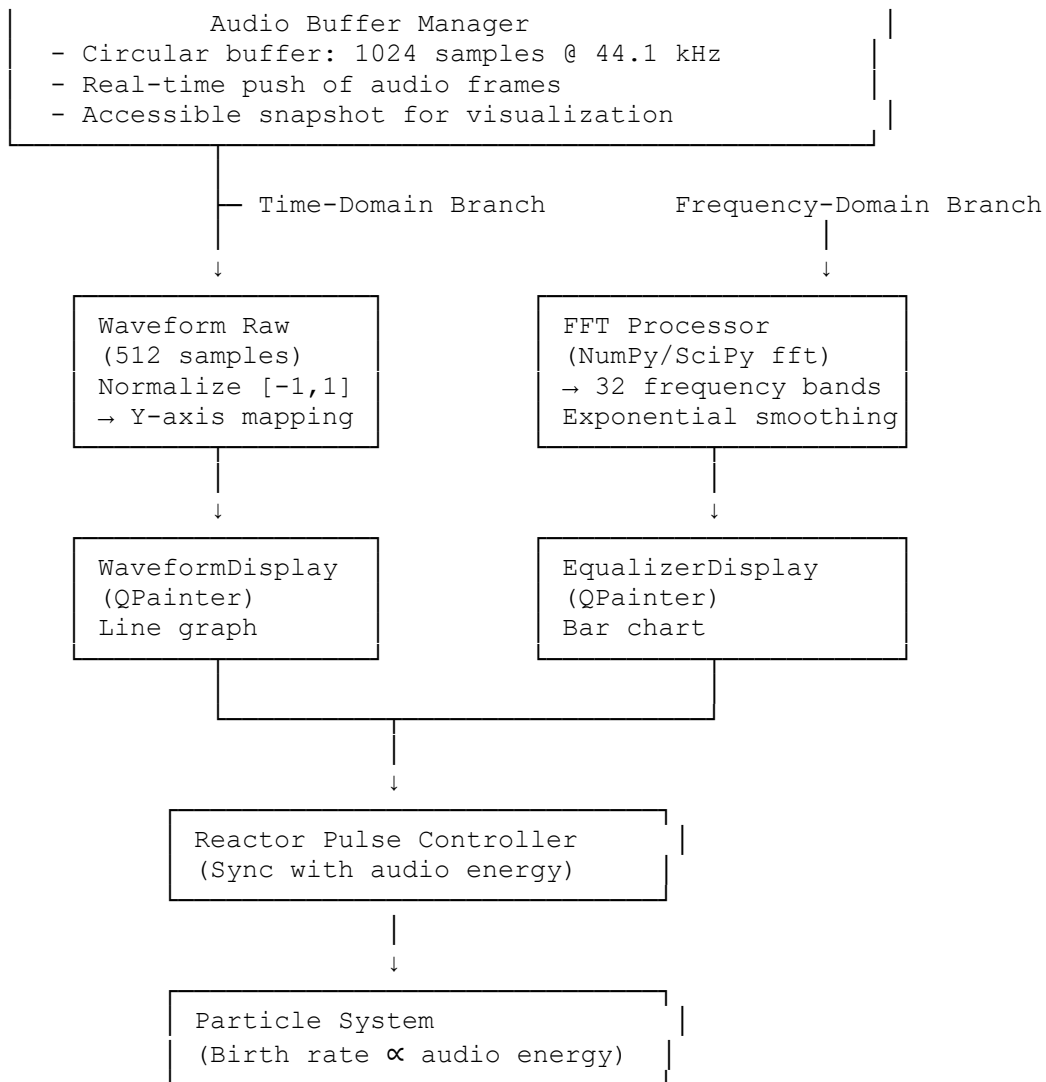
- **Use QPropertyAnimation** for widget property changes (leverages Qt's C++ optimizations)
- **Custom QTimer @ 60 FPS** only for real-time audio visualization
- **Throttle updates** to dirty regions using `update(rect)` instead of full repaints
- **Cache static graphics** as `QPixmap` for rings/backgrounds
- **Lazy-load** heavy computations (FFT) in background threads
- **Profile with PerformanceMonitor** to track FPS and paint times

9. AUDIO-REACTIVE VISUALIZATION DESIGN

9.1 Data Flow: Audio → Visual Properties

Code





9.2 Algorithm: Waveform Extraction

Python

```

def extract_waveform(audio_buffer):
    """
    Args:
        audio_buffer: Circular buffer of audio samples (normalized)

    Returns:
        waveform_points: List of (x, y) tuples for line drawing
    """
    samples = audio_buffer.get_latest(512)  # Last 512 samples

    # Normalize samples to float32 [-1.0, 1.0]
    normalized = np.array(samples, dtype=np.float32) / 32768.0

    # Map to canvas coordinates
    canvas_width = 800
  
```

```

    canvas_height = 200

    points = []
    step = canvas_width / len(normalized)

    for i, amplitude in enumerate(normalized):
        x = i * step
        y = (canvas_height / 2) - (amplitude * canvas_height / 2) # Center-
mapped
        points.append((x, y))

    return points

```

9.3 Algorithm: FFT & Equalizer Bands

Python

```

def extract_frequency_bands(audio_buffer, num_bands=32):
    """
    Args:
        audio_buffer: Circular audio buffer
        num_bands: Number of equalizer bars

    Returns:
        band_magnitudes: List of magnitude values [0, 1] for each band
    """
    samples = audio_buffer.get_latest(2048) # 2048 for better freq
resolution

    # Apply FFT
    fft_output = np.fft.rfft(np.array(samples))
    magnitudes = np.abs(fft_output)

    # Logarithmic frequency scaling (human-perceived pitch)
    log_scale = np.logspace(0, np.log10(len(magnitudes) - 1), num_bands)
    band_indices = log_scale.astype(int)

    # Extract magnitude for each band
    band_mags = magnitudes[band_indices]

    # Normalize to [0, 1]
    normalized = band_mags / (np.max(magnitudes) + 1e-6)

    # Smooth with exponential averaging
    smoothed = exponential_smooth(normalized, alpha=0.3)

    return smoothed

```

9.4 Reactor Pulse Synchronization

Python

```

class AudioReactiveReactor:
    """
    Reactor brightness and rotation speed synchronized with audio energy.
    """

```

```

def __init__(self):
    self.energy_buffer = CircularBuffer(100) # Last 100 frames of energy
    self.base_rotation_speed = 1.0
    self.max_rotation_speed = 3.0

def update_from_audio(self, frequency_bands):
    """
    Calculate overall audio energy and update reactor.
    """
    energy = np.mean(frequency_bands) # Overall energy (0-1)
    self.energy_buffer.push(energy)

    # Map energy to rotation speed
    self.rotation_speed = self.base_rotation_speed + \
        energy * (self.max_rotation_speed -
self.base_rotation_speed)

    # Map energy to glow brightness
    self.glow_brightness = 0.5 + energy * 0.5 # Range [0.5, 1.0]

```

10. STATE MANAGEMENT SYSTEM

10.1 Global State Structure

Python

```

class AppState:
    """
    Central state container. Observable via event_bus.
    """

    def __init__(self):
        # Application
        self.current_screen = "startup" # startup | mode_selection | chat |
voice
        self.is_running = True

        # Voice & Audio
        self.is_listening = False
        self.is_speaking = False
        self.is_processing = False
        self.microphone_level = 0.0 # 0-1
        self.speaker_level = 0.0 # 0-1

        # Ollama
        self.ollama_connected = False
        self.current_model = "mistral"
        self.last_response_time_ms = 0

        # Chat
        self.chat_history = [] # List of (role, message, timestamp)
        self.current_input = ""

```

```

# System
self.system_temperature = 0.0
self.system_time = datetime.now()
self.battery_level = 100

# Audio Visualization
self.waveform_data = [] # Real-time waveform points
self.equalizer_bands = [0.0] * 32 # 32 frequency bands

```

10.2 Event System

Python

```

class EventBus:
    """
    Publish-Subscribe event system for decoupled communication.
    """

    EVENTS = {
        "STATE_CHANGED": "state_changed",
        "SCREEN_CHANGED": "screen_changed",
        "VOICE_STARTED": "voice_started",
        "VOICE_ENDED": "voice_ended",
        "OLLAMA_RESPONSE": "ollama_response",
        "AUDIO_LEVEL_CHANGED": "audio_level_changed",
        "ANIMATION_COMPLETE": "animation_complete",
    }

    def publish(self, event_type, data=None):
        """Broadcast event to all subscribers"""

    def subscribe(self, event_type, callback):
        """Register listener for event"""

    def unsubscribe(self, event_type, callback):
        """Unregister listener"""

```

10.3 State Update Flow

Code

```

User Action (Voice Input / Button Click)
↓
State Manager Updates AppState
↓
Event Bus Publishes Event
↓
Subscribers React
├── UI Layer: Update visuals
├── Animation Layer: Trigger animations
├── Integration Layer: Call Ollama / Voice handlers
├── Audio Layer: Update visualization
↓
New State Published
↓
UI Re-renders (if needed)

```

11. VOICE INTEGRATION STRATEGY

11.1 Voice Handler Wrapper

Python

```
class VoiceHandler:
    """
    Wrapper for existing voice input/output system.
    Provides unified interface for new UI layer.
    """

    def __init__(self, existing_voice_system):
        self.voice_system = existing_voice_system
        self.event_bus = EventBus()

    def start_listening(self):
        """Begin capturing microphone input"""
        # Delegate to existing system
        # Publish VOICE_STARTED event

    def stop_listening(self):
        """Stop capturing microphone input"""
        # Publish VOICE_ENDED event

    def get_recognized_text(self):
        """Retrieve STT result from existing system"""

    def speak(self, text, voice_profile="jarvis"):
        """
        Generate and play speech using TTS.

        Args:
            text: String to speak
            voice_profile: "jarvis" (deepened), "default", etc.
        """
        # Apply JARVIS voice processing
        # Delegate to existing TTS system
        # Stream audio to visualization
        # Publish VOICE_ENDED on completion

    def get_microphone_level(self):
        """Real-time microphone amplitude (0-1)"""

    def get_speaker_level(self):
        """Real-time speaker amplitude (0-1)"""
```

11.2 JARVIS Voice Deepening

Python

```
class JARVISVoiceProcessor:
```

```

"""
Apply JARVIS-style voice characteristics to TTS output.
"""

JARVIS_PROFILE = {
    "base_pitch": -2.0,          # Lower pitch (semitones)
    "pitch_variance": 0.3,      # Subtle pitch modulation
    "speaking_rate": 1.0,       # Normal speed
    "voice_id": "en_male_formal", # British male (if available)
    "timbre_warmth": 0.7,       # Warm tone (0-1)
    "clarity_emphasis": 0.9,    # Crisp articulation
    "reverb_amount": 0.15,     # Subtle space/echo
    "resonance_boost": [200, 400], # Boost certain frequencies (Hz)
}

def apply_voice_profile(self, audio_data, profile=JARVIS_PROFILE):
    """
    Process audio to apply JARVIS characteristics.

    Returns:
        processed_audio: Audio with JARVIS voice applied
    """
    # Lower pitch using time-stretch or pitch-shift
    # Apply EQ for resonance boosts
    # Add subtle reverb
    # Ensure crisp articulation

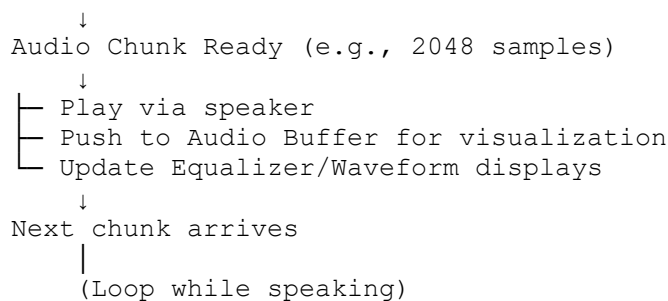
def get_tts_parameters(self):
    """
    Return TTS service parameters (ElevenLabs, etc.).
    """
    return {
        "voice_id": "en_male_formal",
        "stability": 0.75, # High stability for consistency
        "similarity_boost": 0.85,
        "style": 0.0, # Neutral, professional
    }

```

11.3 Real-Time Audio Streaming

Code

Voice Output Generation

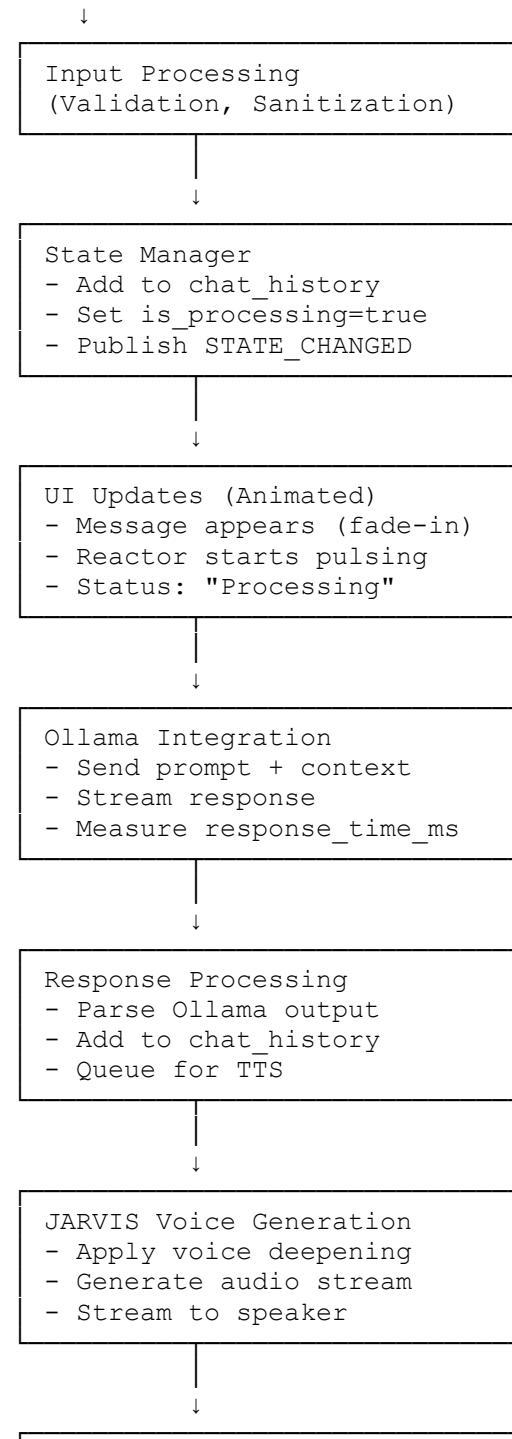


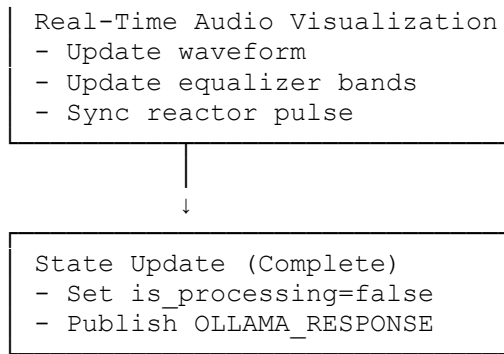
12. DATA FLOW DIAGRAMS

12.1 Chat Message Flow

Code

User Input (Text or Voice)





12.2 Voice Input Flow

Code

```
Microphone Stream
├── Captured by Voice Handler
├── Pushed to Audio Buffer
├── Update Waveform Display (real-time)
├── Update Equalizer Display (FFT @ 60 Hz)
├── Update Microphone Level Meter
├── (STT processing in background)
└── Recognized Text Ready
    └── Feed to Chat Message Flow (above)
```

12.3 Voice Output Flow

Code

```
JARVIS Speaking
├── Audio stream available
├── Play via speaker
├── Push chunks to Audio Buffer
├── Update Waveform Display (JARVIS voice waveform)
├── Update Equalizer (JARVIS voice spectrum)
├── Update Speaker Level Meter
├── Sync Reactor Pulse (brightness ∝ audio energy)
├── (Audio playback complete)
└── Update State → is_speaking=false
```

13. PERFORMANCE OPTIMIZATION STRATEGY

13.1 Rendering Performance

Optimization	Method	Expected Benefit
Reduce Draw Calls	Use <code>update(rect)</code> for dirty regions only	30-40% reduction in repaints
Cache Static Graphics	Pre-render rings/backgrounds as QPixmap	50% faster rendering of Startup Screen
GPU Acceleration	Use QOpenGLWidget for complex visualizations	2-3x faster for equalizer with 32 bars
Throttle Updates	Cap animation timer @ 60 FPS (16.7 ms/frame)	Consistent frame rate
Lazy Load Content	Load chat history incrementally	Faster initial UI show

13.2 CPU Performance

Optimization	Method	Expected Benefit
Async FFT	Compute FFT on background thread	No UI jank during audio analysis
Smooth Averaging	Use exponential averaging (not full FFT every frame)	70% reduction in FFT calls
Object Pooling	Reuse animation objects instead of creating new	Reduced GC pressure
Profile Hotspots	Use <code>cProfile</code> to identify slow functions	Focus optimization effort

13.3 Memory Performance

Optimization	Method	Expected Benefit
Audio Buffer Size	Circular buffer of 4 KB (1024 samples @ 16-bit)	Low memory footprint
Chat History Pagination	Load last 50 messages, lazy-load older	Constant memory regardless of history size
Font Caching	Load fonts once at startup	Avoid repeated disk I/O
Image Caching	Store theme colors in constants	Reduced color object allocations

13.4 Profiling Tools

Python

```
class PerformanceMonitor:
    """Debug tool for tracking FPS and paint times."""

    def __init__(self):
        self.frame_times = deque(maxlen=60) # Last 60 frames
        self.fft_times = deque(maxlen=60)

    def record_frame_time(self, ms):
        """Record time taken for one frame"""

    def record_fft_time(self, ms):
        """Record FFT computation time"""

    def get_average_fps(self):
        """Calculate average FPS from frame times"""

    def log_stats(self):
        """Log FPS, FFT time, memory usage"""
```

14. DATE/TIME BUG FIX STRATEGY

14.1 Current Bug

Issue: JARVIS says "I cannot access the current date/time" instead of fetching from system.

Root Cause: Voice output handler doesn't query system time; chat integration doesn't inject time context into Ollama prompts.

14.2 Fix Implementation

Step 1: Create Dedicated Date/Time Handler

Python

```
class DateTimeHandler:
    """
    Unified system time provider with caching and timezone support.
    """

    def __init__(self, timezone="auto"):
        self.timezone = self._detect_timezone() if timezone == "auto" else timezone
        self.last_update = None
        self.cache_duration_ms = 1000 # Refresh every 1 second

    def get_current_time(self, format_str="%I:%M %p"):
        """
```

```

    Get current time with timezone awareness.

    Returns:
        str: Formatted time (e.g., "02:30 PM")
    """
    now = datetime.now(tz=self.timezone)
    return now.strftime(format_str)

def get_current_date(self, format_str="%A, %B %d, %Y"):
    """
    Get current date.

    Returns:
        str: Formatted date (e.g., "Saturday, May 30, 2026")
    """
    now = datetime.now(tz=self.timezone)
    return now.strftime(format_str)

def _detect_timezone(self):
    """Auto-detect system timezone"""
    import tzlocal
    return tzlocal.get_localzone()

```

Step 2: Inject Time Context into Ollama Prompts

Python

```

class ChatManager:
    """Enhanced chat manager with time context."""

    def __init__(self, ollama_client, datetime_handler):
        self.ollama = ollama_client
        self.datetime_handler = datetime_handler

    def build_prompt_with_context(self, user_message):
        """
        Build prompt with system context (time, date, etc.).
        """
        current_time = self.datetime_handler.get_current_time()
        current_date = self.datetime_handler.get_current_date()

        system_context = f"""
You are JARVIS, an advanced AI assistant.
Current Date: {current_date}
Current Time: {current_time}
When asked about the time or date, refer to the above.
Maintain a calm, professional, and intelligent demeanor.
"""

        return system_context, user_message

    def send_message(self, user_message):
        """
        Send message to Ollama with time context.
        """
        system_context, prompt = self.build_prompt_with_context(user_message)

```

```

# Call Ollama with system context
response = self.ollama.generate(
    model=self.current_model,
    prompt=prompt,
    system=system_context,
    stream=True
)

return response

```

Step 3: Update Status Bar with Live Time

Python

```

class StatusBar(QStatusBar):
    """Status bar showing live time, temp, battery."""

    def __init__(self, datetime_handler):
        super().__init__()
        self.datetime_handler = datetime_handler

        # Timer to update time every second
        self.update_timer = QTimer()
        self.update_timer.timeout.connect(self.update_status)
        self.update_timer.start(1000)

    def update_status(self):
        """Update time and system stats"""
        time_str = self.datetime_handler.get_current_time()
        temp = self.get_system_temperature()
        battery = self.get_battery_level()

        status_text = f" {time_str} | Temp: {temp}°C | Battery: {battery}%"
        self.showMessage(status_text)

```

Step 4: Test Time Injection

- When user asks "What time is it?", Ollama should respond with current system time.
- When user asks "What's the date?", Ollama should respond with current system date.
- Status bar updates live every second.

15. DEEPENING THE JARVIS VOICE (TOP PRIORITY)

15.1 JARVIS Voice Characteristics (Reference)

Based on canonical JARVIS (Paul Bettany) from Iron Man:

Characteristic	Target	Implementation
Pitch	Mid-to-low masculine	-2 to -4 semitones below baseline
Tone	Warm, calm, authoritative	EQ boost @ 200-400 Hz (warmth)
Articulation	Crisp, precise, clear	EQ boost @ 2-4 kHz (clarity)
Accent	Subtle British English	Select British voice in TTS
Speed	Natural, conversational	1.0x speed (not rushed)
Formality	Professional, respectful	Neutral prosody (no emotional affect)
Digitality	Subtle synthetic undertone	Light reverb (15-20% wet) + slight presence peak
Resonance	Deep, commanding	Boost lower mids (400-600 Hz)

15.2 Voice Deepening Algorithm

Python

```
class JARVISVoiceDeepener:
    """
    Process audio or TTS parameters to achieve JARVIS voice.
    """

    # EQ Profile for deepening
    JARVIS_EQ_PROFILE = {
        "low_shelf": {"freq": 200, "gain": +3, "q": 0.7},      # Warmth
        "mid_boost_1": {"freq": 400, "gain": +2, "q": 1.0},   #
Body/resonance
        "mid_boost_2": {"freq": 600, "gain": +1.5, "q": 1.0}, # Presence
        "presence_peak": {"freq": 3000, "gain": +2, "q": 1.5}, # Clarity
        "high_shelf": {"freq": 8000, "gain": -1, "q": 0.7},   # Reduce
harshness
    }

    # Reverb for subtle space
    JARVIS_REVERB = {
        "room_size": 0.3, # Small room (subtle)
        "wet": 0.15,      # 15% wet signal
        "dry": 0.85,      # 85% dry
        "damping": 0.5,
    }

    def deepen_audio(self, audio_samples, sample_rate):
        """
        Apply JARVIS voice characteristics to raw audio.

        Args:
            audio_samples: NumPy array of audio samples
            sample_rate: Sample rate (e.g., 44100 Hz)
        """
```

```

Returns:
    deepened_audio: Processed audio with JARVIS voice
"""
# 1. Apply EQ (10-band parametric EQ)
deepened = self._apply_eq(audio_samples, sample_rate)

# 2. Apply reverb (using convolution or algorithmic reverb)
deepened = self._apply_reverb(deepened)

# 3. Normalize to prevent clipping
deepened = self._normalize(deepened)

return deepened

def _apply_eq(self, audio, sample_rate):
    """Apply parametric EQ filters"""
    # Use scipy.signal.iirfilter for each band
    # Apply filters sequentially
    # Return filtered audio
    pass

def _apply_reverb(self, audio):
    """Apply reverb effect"""
    # Use convolution with impulse response
    # Or use algorithmic reverb (Schroeder reverberator)
    pass

def _normalize(self, audio):
    """Normalize to [-1.0, 1.0] to prevent clipping"""
    max_val = np.max(np.abs(audio))
    if max_val > 0:
        audio = audio / max_val
    return audio

def get_tts_voice_parameters(self):
    """
    Return parameters for TTS service (ElevenLabs, Google Cloud TTS,
    etc.).

    Returns:
        dict: Voice parameters for TTS API
    """
    return {
        "voice_id": "en_male_formal", # Or similar British male voice
        "pitch": -2.0,                # Slightly lower pitch
        "speed": 1.0,                  # Normal speed
        "stability": 0.75,              # High stability for consistency
        "similarity_boost": 0.85,       # Balance between voice similarity
        "style": 0.0,                  # Neutral style (professional)
    }

```

15.3 TTS Integration Points

Python


```

class VoiceOutputManager:
    """
    Manages JARVIS voice output with deepening applied.
    """

    def __init__(self, tts_provider, voice_deepener):
        self.tts = tts_provider # Existing TTS system
        self.deepener = voice_deepener

    def speak_as_jarvis(self, text, use_deepening=True):
        """
        Generate and play JARVIS voice.

        Args:
            text: String to speak
            use_deepening: Apply voice deepening (True by default)
        """
        # Get TTS voice parameters for JARVIS
        voice_params = self.deepener.get_tts_voice_parameters()

        # Generate audio from TTS provider
        audio_stream = self.tts.synthesize(text, **voice_params)

        # If deepening enabled and using local TTS, apply post-processing
        if use_deepening and isinstance(audio_stream, np.ndarray):
            audio_stream = self.deepener.deepen_audio(audio_stream,
sample_rate=44100)

        # Play audio
        self._play_audio(audio_stream)

        # Stream to visualization
        self._stream_to_visualizer(audio_stream)

    def _play_audio(self, audio):
        """Play audio to speaker"""
        # Use existing playback system
        pass

    def _stream_to_visualizer(self, audio):
        """Push audio to visualization engine"""
        # Update waveform & equalizer
        pass

```

15.4 Voice Testing & Iteration

Code

Test Scenarios:

1. Startup Greeting: "Welcome back, Samar."
2. Query Response: Short, factual response (~20 words)
3. Long Response: Extended explanation (~100 words)
4. Status Report: System diagnostics read-out

Evaluation Criteria:

✓ Pitch: Deep, masculine, mid-range

- ✓ Tone: Calm, professional, warm
- ✓ Clarity: Words are crisp and well-articulated
- ✓ Consistency: Same voice across all phrases
- ✓ No artifacts: No clipping, distortion, or robotic quality

SUMMARY TABLE: Architecture Components

Component	Type	Priority	Location
State Manager	Core Logic	CRITICAL	core/state_manager.py
Event Bus	Communication	CRITICAL	core/event_bus.py
Animation Engine	UI	HIGH	animations/animation_engine.py
Startup Screen	UI	HIGH	ui/screens/startup_screen.py
Chat Dashboard	UI	HIGH	ui/screens/chat_dashboard.py
Voice Dashboard	UI	HIGH	ui/screens/voice_dashboard.py
Arc Reactor Widget	UI	HIGH	ui/widgets/arc_reactor.py
Waveform Visualizer	Audio Viz	HIGH	ui/widgets/waveform_display.py
Equalizer Visualizer	Audio Viz	HIGH	ui/widgets/equalizer_display.py
JARVIS Voice Processor	Audio	HIGH	audio/jarvis_voice_processor.py
Voice Handler Wrapper	Integration	CRITICAL	audio/voice_handler.py
Date/Time Handler	Utility	HIGH	integration/date_time_handler.py
Ollama Client	Integration	CRITICAL	integration/ollama_client.py
Chat Manager	Logic	HIGH	integration/chat_manager.py